Certify-or-Grow and Lazy SF-FSA-VI in stochastic domains: analysis

October 25, 2026

This note collects the evidence behind Sections 4–5 of the paper (`paper/article.tex`): the cost of building the CCS with Certify-or-Grow (CoG) under noise (§1), and Lazy SF-FSA-VI with exact SFs (§2) and with learned SFs against the previous algorithm, SF-FSA-VI over the policies of SFOLS [1], and against Q-learning (§3). Code, data and commands: `HANDOFF.md`.

Summary.

- **CCS under noise.** The sample gain of CoG over SFOLS is a deterministic-domain effect. In the double slit (wind ±1) CoG is as fast as warm-started SFOLS (and 3× faster than SFOLS); in slippery Office it is 9% slower with the model certificate and 11% faster with the model-free one, which false-certifies 16% of the metapolicies.
- **Lazy SF-FSA-VI with exact SFs** is optimal on all 126 tasks with about 2 added policies per task; keeping them, a new task is zero-shot (no policy added) in 58–92% of the cases, 71–97% in the second half of the sequence.
- **Without a model** (oracle certificate on policies and estimates), Lazy SF-FSA-VI brings every windy task within 0.4% of optimal (zero-shot over the learned extremes: up to 2.1%) with about one new policy and 1M steps per task; once a few policies are stored, 87–97% of new tasks need none. It needs 2–4× fewer samples than SF-FSA-VI over SFOLS (20–260× after the shared extremes), whose plan even degrades as noisy tables accumulate. Growing with the metapolicy costs as much as Q-learning over all actions. Q-learning on the product MDP is 120× more expensive per task in windy, but in Office it is cheaper than learning the extremes (they pay off after 85–155 tasks).

1 Certify-or-Grow for the CCS in stochastic domains

Protocol of the paper's Figure 1 (`q_ccs_stochastic.py`, 10 seeds): SF Q-learning with step $1/(n+1)^{0.7}$, ϵ-greedy 0.1, exploring starts, tight optimistic start; a learn stops when its greedy policy is unchanged for 30 checks of 1000 steps (and every pair has 30 visits) or at the step cap (3M Office, 5M double slit); "steps to converge" is the last check where the greedy value at the start is more than 10^{-3} below the optimum (oracle). Certificate tolerance 10^{-2}; the model-free version completes the pairs outside $B(s)$ with 100 samples each. CoG learns its 2nd..d-th extreme warm-started, as SFOLS + warm start (`QBASISWARM=1`).

Table 1: Model-free CCS construction under noise: mean ± std over 10 seeds (Figure `q_ccs_cost_stochastic.pdf`). Office: slip 0.05, ε-CCS with $\varepsilon = 0.05$. Double slit: wind ±1, $\gamma = 0.99$, $\varepsilon = 10^{-3}$.

	Office, slip 0.05		Double slit, wind ±1	
	to converge	all steps	to converge	all steps
SFOLS	135.7 ± 5.6M	191.6 ± 8.9M	13.2 ± 1.3M	26.0 ± 2.0M
SFOLS + warm start	133.4 ± 5.4M	193.2 ± 9.4M	4.24 ± 0.66M	30.6 ± 2.8M
CoG + warm start (model certificate)	147.4 ± 12.6M	206.5 ± 14.5M	4.49 ± 1.36M	21.3 ± 2.7M
CoG + warm start (model-free certificate)	120.9 ± 10.9M	176.6 ± 15.4M	4.94 ± 1.59M	22.5 ± 3.8M

- **Double slit.** After the basis, the metapolicies are certified (5.2 of 5.4 queries) and converge almost immediately; the basis is most of the cost. CoG matches warm-started SFOLS in steps to converge and stops 30% earlier (21M against 31M steps). With 3 seeds CoG looked 12% faster than warm SFOLS (HANDOFF E5); with 10 it is not.

- **Office.** The learned metapolicies are noisy: the exact gap $\leq 10^{-2}$ fails on 17.5 of 96.5 queries, which then pay the metapolicy and a full learn (+9% against SFOLS). The model-free certificate falls back less (5.9) and is 11% cheaper, but 15.5 of its 94 certificates are false (true gap above the tolerance).
- **Why.** Under noise the cost is learning SFs accurately, not deciding the control: the metapolicy restricts the actions ($|B(s)| \approx 2.9$ of 4 in Office) but its entries must still be learned to the noise floor.

2 Lazy SF-FSA-VI with the model

Problem. SF-FSA-VI [1] plans a task automaton over a set of base policies with known SFs. Entering exit j in automaton state u moves to $\tau(u, j)$: the accepting state T, another state v, or u itself when the exit's symbol has no edge from u. Exit weights solve

$$w_j(u) = \begin{cases} 1 & \tau(u,j) = T \\ \gamma\, V_{\mathbf{w}(\tau(u,j))}(\varepsilon_j) & \text{otherwise,} \end{cases}$$

which Kuric et al. iterate with $V_{\mathbf{w}}(\varepsilon_j) = \max_{a,\pi} \boldsymbol{\psi}^{\pi}(\varepsilon_j, a) \cdot \mathbf{w}$ (GPI estimates at the exits). *Assumption:* the jump into the next automaton state is discounted by γ (entering an exit takes a step), so the iteration is a γ-contraction even when an exit can be re-entered at once. The iteration is optimal if the base is a CCS. In stochastic domains the CCS is large (117–572 policies for the ϵ-CCSs below) and an ϵ-CCS is not exact.

Idea. The planner only reads values at the weights $\mathbf{w}(u)$ it visits, and these converge. So instead of a CCS, start from the d extremes and add a policy only at a visited weight where the base is provably not exact. *Certificate:* the GPI estimate $\hat{Q}_{\mathbf{w}} = \max_j Q^{\pi_j}_{\mathbf{w}}$ is a lower bound with $T\hat{Q}_{\mathbf{w}} \geq \hat{Q}_{\mathbf{w}}$, so $T\hat{Q}_{\mathbf{w}} = \hat{Q}_{\mathbf{w}}$ (one lookahead) implies $\hat{Q}_{\mathbf{w}} = Q^*_{\mathbf{w}}$. *Growth:* the metapolicy over the base first (composition), then π^{hyb}, then improvement [2].

Algorithm 1 Lazy SF-FSA-VI

Require: base L (the d extremes, or the base left by earlier tasks), task automaton

repeat
  $W \leftarrow$ fixed point of Kuric's iteration over L (warm-started) ▷ zero-shot; no low-level solve
  for every automaton state u **do**
    if $T\hat{Q}_{\mathbf{w}(u)} = \hat{Q}_{\mathbf{w}(u)}$ **then continue** ▷ certified: GPI over L is exact at $\mathbf{w}(u)$
    end if
    $\pi \leftarrow$ metapolicy over L at $\mathbf{w}(u)$ ▷ composition of stored policies, actions $B(s)$
    if the Bellman gap of π is not zero **then** $\pi \leftarrow \pi^{\text{hyb}}$, then improve until the gap is zero
    end if
    $L \leftarrow L \cup \{\pi\}$ (its SFs) ▷ one small refinement
  end for
until no automaton state needed a new policy
execute GPI over L at $\mathbf{w}(u)$ in every u

Theorem 1. *(i) Every W computed is a lower bound on the optimal weights W^*. (ii) Algorithm 1 terminates. (iii) At termination $W = W^*$ and the executed policy is optimal in the product MDP. (iv) In deterministic domains it adds nothing.*

Proof. (i) GPI estimates are lower bounds and the iteration is monotone. (ii) An uncertified $\mathbf{w}(u)$ has no optimal policy in L (it would make $\hat{Q}_{\mathbf{w}(u)} = Q^*_{\mathbf{w}(u)}$), so each added policy is new; there are finitely

Table 2: With the model. Kuric et al.'s plan over the 6 extremes and over the full ϵ-CCS of SFOLS (Office: $\epsilon = 2 \cdot 10^{-3}$, windy: 10^{-5}), and the shortest OLS-order prefix of that CCS that makes it optimal (executed relative regret $\leq 10^{-9}$); Algorithm 1 from the 6 extremes on every task (fresh) or with the base carried over the task sequence (stream). Kinds: policies certified as the metapolicy / π^{hyb} / after policy iteration. Cost in full sweeps per task (SF tables of added policies included); the baseline solves the product MDP by value iteration from scratch, without SFs. Planning: $|\mathcal{U}||\mathcal{E}||\Pi|\times$ iterations of Kuric's update per task. 33 Office tasks (the 3 of [1] and 30 random), 60 windy tasks.

	Office, slip 0.1	Office, slip 0.3	Windy two-stage
Kuric, 6 extremes: max relative regret	$1.1 \cdot 10^{-3}$	$3.2 \cdot 10^{-3}$	$1.4 \cdot 10^{-2}$
Kuric, full ϵ-CCS: max relative regret (size)	$3.1 \cdot 10^{-6}$ (262)	$2.5 \cdot 10^{-4}$ (572)	0 (117)
CCS prefix that is optimal: median (never optimal)	– (17 of 33)	100 (5 of 33)	11–15 (0)
Lazy, regret	0	0	0
fresh: rounds / policies added per task	2.15 / 1.79	2.61 / 2.42	2.00 / 2.20
stream: policies added per task / final base	0.67 / 28	0.73 / 30	0.12 / 13
stream: tasks with no policy added (second half)	58% (71%)	58% (71%)	92% (97%)
stream: certificate checks / added, per task	2.8 / 0.67	3.0 / 0.73	3.1 / 0.12
kinds, fresh: metapolicy / π^{hyb} / PI	10 / 22 / 27	31 / 14 / 35	24 / 81 / 27
kinds, stream: metapolicy / π^{hyb} / PI	18 / 2 / 2	19 / 4 / 1	3 / 4 / 0
cost per task: lazy fresh / lazy stream	200 / 72	558 / 160	229 / 24
cost per task: product MDP from scratch	67	130	282
planning per task: lazy stream / Kuric over the ϵ-CCS	$4.7 \cdot 10^3$ / $4.7 \cdot 10^4$	$9.6 \cdot 10^3$ / $1.8 \cdot 10^5$	$6.0 \cdot 10^3$ / $5.9 \cdot 10^4$

many deterministic policies. (iii) At the stop every $\mathbf{w}(u)$ is certified, so the GPI estimates at the exits are $V^*_{\mathbf{w}(u)}$: $W = T_L W = T^* W$, whose unique fixed point is W^*; GPI is greedy for $Q^*_{\mathbf{w}(u)}$ in every u. (iv) The d extremes give $\hat{Q}_{\mathbf{w}} = Q^*_{\mathbf{w}}$ for every $\mathbf{w}$ [3]. □

Results (Table 2, paper Figure 4).

- **Exact with a handful of policies.** Algorithm 1 is optimal on every task in 2–2.6 rounds, adding about 2 policies per task. With the base carried over, it adds 0.12–0.73 per task and ends with 13–30 policies, where Kuric's plan needs 11–15 CCS policies per windy task, a median of 100 at slip 0.3, and is never optimal on half of the slip-0.1 tasks even with all 262. In deterministic Office it adds nothing (Theorem, (iv)).
- **It becomes zero-shot.** From the extremes no task passes the exact certificate as it is (a stochastic optimum always needs some hedging). Keeping the added policies, 58% of the Office tasks and 92% of the windy ones are certified without adding anything, 71% and 97% in the second half; a task then costs about 3 certificate checks.
- **Refinements are compositions.** With a growing base, 91–100% of the added policies are the metapolicy or π^{hyb} over stored policies, with no improvement step over all actions (54–80% from the extremes; in windy mostly π^{hyb}, the hedging step). Growing by GPI and policy iteration instead adds the same number of policies, but only 8–71% are the GPI policy itself.
- **Compute.** With the model the lazy plan costs about as much as solving the product MDP in Office (the metapolicy does not save sweeps over policy iteration), and 12× less in windy once the base has grown. Planning (Kuric's iteration) is 10–18× cheaper than over the ϵ-CCS, as $|\Pi| \leq 30$ instead of 117–572.

3 Lazy SF-FSA-VI without the model

Protocol (`q_lazy_fsa.py`, numba learners of `mf.py`, 10 seeds, the task sets of Table 2). Every policy, the d extremes included, is learned by SF Q-learning (step $1/(n+1)^{0.7}$, ϵ-greedy 0.1, exploring starts); the model only measures. All methods share the learned extremes.

- *Lazy SF-FSA-VI* (Algorithm 1). The certificate is replaced by an oracle check (*assumption*) that, at $\mathbf{w}(u)$, the GPI policy and the GPI estimate are within δ of $V^*_{\mathbf{w}(u)}$ at the start and the exits. Growth: *metapolicy* (model-free CoG) learns on $B(s)$ until its policy has converged and keeps it if it is optimal, otherwise continues over all actions; *Q-learning* learns over all actions. Both start at the GPI table. At most 5 rounds per task.
- *SF-FSA-VI over SFOLS* [1]: OLS corner weights (priority by the improvement bound, $\epsilon = 10^{-3}$, up to 100 queries) from the same extremes, every query learned from the GPI table; after every query, Kuric's plan over all the learned tables is evaluated on every task.
- *Q-learning on the product MDP* (Kuric's Flat-Q), from scratch on every task, exploring starts over $\mathcal{U} \times \mathcal{S}$.
- Learners stop when their greedy policy (and, for SFs, their estimated value) has been within the tolerance of its optimum for 100 checks of 1000 steps in a row, or at 50M steps (300M for Flat-Q). Tolerances relative to $\max V^*_{\mathbf{w}}$ at the start and exits: 10^{-3} for policies, 10^{-2} for estimates; Flat-Q: $10^{-3}V^*$ at the start.

What the pilots taught (`cluster/sweeps/lazy_mf_pilot*.yaml`, 8 tasks, 1 seed).

- **The certificate must check the estimates.** Checking only the GPI policy at $\mathbf{w}(u)$ certified windy tasks whose plan had 15% regret: Kuric's iteration reads the *estimates* at the exits, and learned ones can be badly biased (optimistic start: 4% regret zero-shot; pessimistic: 16%). Algorithm 1's certificate is a statement about the estimate, so the oracle version must check it too; with it, every windy task ends within 0.4%.
- **Pessimistic start.** SF tables start at zero, so learned estimates are biased low and GPI stays an approximate lower bound, as with exact SFs; an optimistic start ($\Psi = \mathbf{e}_i$) takes 7–26M steps to unlearn in windy corridors that cannot reach the exit (0.2–0.4M from zero).
- **Estimate tolerance.** Requiring estimates within 10^{-3} at every exit is beyond the noise floor of $1/(n+1)^{0.7}$ steps in Office (extremes capped at 50M); 10^{-2} removes the gross biases that break the plan.
- **Inherited counts** (warm starts that keep the visit counts of the copied entries, HANDOFF E6) make new policies $\sim 10\times$ slower when the new policy differs from the stored ones (hedging), because the step sizes start tiny; SFOLS, whose queries are close to stored policies, benefits from them in Office and windy. Each method is reported with its better setting: Lazy without inherited counts (4–14× cheaper, same regret), SFOLS with them in windy and Office (cheaper; in windy its plan degrades without them, below) and without them in the double slit.

Results (Table 3, Figure 1).

- **How long a new task takes.** From the extremes: 0.7–1.3 new policies and 0.2–2.3M steps per task (one policy costs 0.3–2.2M steps, median). With the policies kept: 0.03–0.15M steps per task and 2–3 certificate checks.
- **It stays zero-shot.** Kept policies make 87–97% of the new tasks certified without learning (91–100% in the second half of the sequence), as with exact SFs (Table 2). In windy, 80% of the new policies are the hedging one at u_0, which differs from the nearest stored policy at ~ 220 of 611 states: it is new behaviour, not a composition (in Office, a median of 48 of 109 states).

Table 3: Without a model: 10 seeds, the task sets of Table 2 (double slit: Kuric et al.'s "blue or red"). Regrets are relative, at the start state, mean / max over tasks and seeds. "Kept": the added policies are carried over the task sequence. Steps per task exclude the extremes, which every SF method shares.

	Windy	Office, $p = 0.1$	Office, $p = 0.3$	Double slit
6 (2) learned extremes: steps	42.6M	128M	231M	33.6M
zero-shot over them: regret	0.90 / 2.1%	0.008 / 0.15%	0.054 / 0.33%	2.1 / 2.3%
Lazy, metapolicy: new policies, steps per task	0.85, 1.07M	0.68, 0.32M	1.31, 1.03M	1.00, 2.20M
regret	0.17 / 0.32%	0.008 / 0.11%	0.014 / 0.12%	0.024 / 0.065%
Lazy, Q-learning: new policies, steps per task	0.83, 1.04M	0.68, 0.23M	1.34, 0.82M	1.00, 2.26M
regret	0.17 / 0.38%	0.009 / 0.11%	0.011 / 0.11%	0.027 / 0.069%
kept: tasks with no new policy (second half)	97% (100%)	91% (95%)	87% (91%)	–
kept: steps per task, metapolicy / Q-learning	0.043 / 0.047M	0.045 / 0.033M	0.151 / 0.135M	–
metapolicy certified alone: from extremes / kept	19 / 41%	8 / 33%	11 / 58%	0%
SF-FSA-VI over SFOLS: steps after the extremes	61.8M	384M	646M	40.7M
regret (queries)	0.21 / 0.50% (100)	0.004 / 0.11% (100)	0.021 / 0.27% (100)	0.012 / 0.045% (8)
Q-learning on the product MDP: steps per task	127M (96% conv.)	0.87M	2.85M	2.07M
total steps, SFOLS / Lazy (metapolicy, kept)	2.3×	4.0×	3.7×	2.1×
tasks until the SFs beat Q-learning on the product	< 1	155	85	–

- **Metapolicy against Q-learning over all actions.** About the same cost: 0.93–1.4× the steps of Q-learning (both from the GPI table), more in Office, where the restricted phase is wasted when it is not certified. The metapolicy is certified alone for 8–19% of the new policies from the extremes and 33–58% with kept policies (a richer $B(s)$), but under noise restricting the actions does not make learning cheaper, as for the CCS (§1).
- **Against SF-FSA-VI over SFOLS.** SFOLS reaches a similar precision with 40–650M steps on top of the extremes (20–260× Lazy's) and stores 100 policies (8 in the double slit) instead of 1–7. Without inherited counts its plan gets *worse* as the set grows (windy: mean regret $\sim 10^{-3}$ after 9–17 queries, then 1.3–9.2% after 62–100): the maximum over many noisy estimates overestimates the exit values. Lazy only adds tables where a task fails the certificate, so it does not accumulate this bias.
- **Against Q-learning on the product MDP.** In windy, 127M steps per task (120× Lazy from the extremes, $\sim$ 3000× with kept policies) and 4% of the tasks do not converge in 300M. In Office it needs only 0.9–2.9M steps per task, while learning the extremes accurately at every exit costs 128–231M: the SFs pay off after 85–155 tasks. In the double slit (one task) both need about 2M steps per task.
- **The cost is the extremes.** As for the CCS, learning SFs accurately is what costs under noise: the shared extremes are 94–99% of Lazy's total over the sequence.

Limits. The certificate needs the model or a simulator; with learned SFs it holds only up to their noise. A sound model-free certificate (confidence bounds from the visit counts) is the main open problem. Termination is guaranteed with exact SFs, but the number of rounds is not bounded in terms of d; without a model it is capped.

References

[1] D. Kuric, G. Infante, V. Gómez, A. Jonsson, H. van Hoof. Planning with a learned policy basis to optimally solve complex tasks. ICAPS 2024.

[2] Certify-or-Grow, `paper/article.tex`.

[3] A small SF basis for SF-FSA-VI, `notes/gpi_basis.tex`.

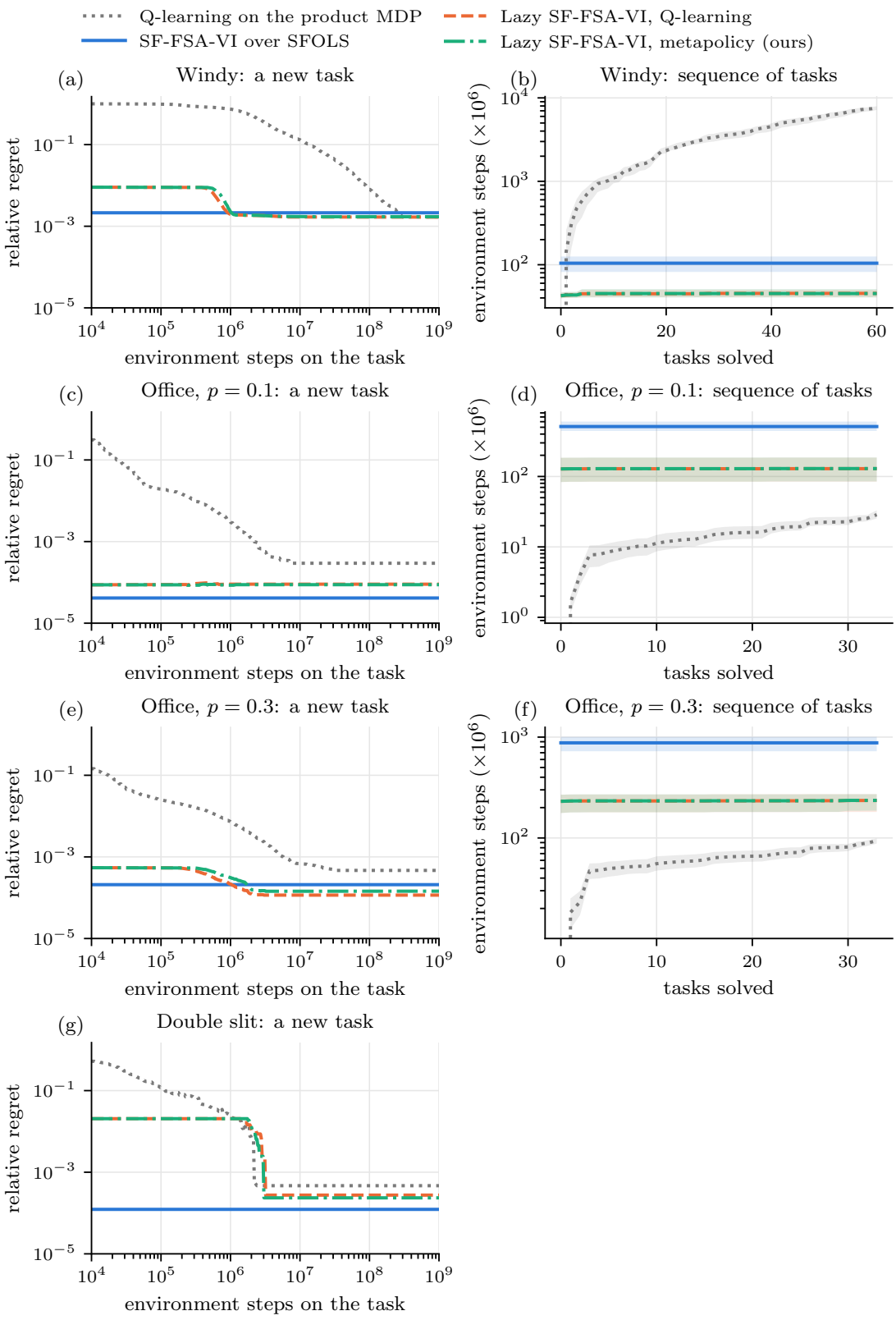


Figure 1: Without a model, all four domains (paper Figure 5 shows windy and Office $p = 0.3$). Left: a new task, relative regret against the steps spent on it (Lazy from the extremes). Right: cumulative steps over the task sequence, extremes included, policies kept.